

Comparative Analysis Report

Structural Comparison:

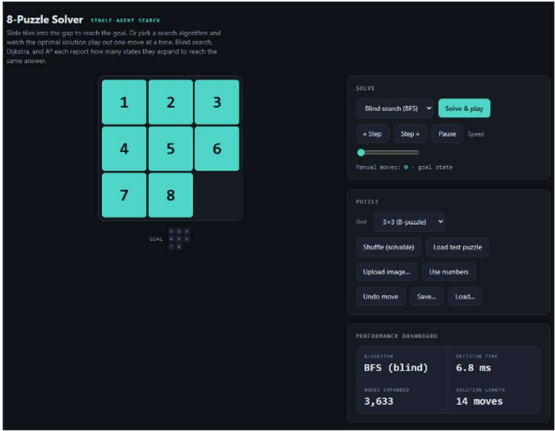
The 8-puzzle and Tic-Tac-Toe both require searching through possible states. But the structure of the search spaces is different. In the 8-puzzle, each state is a board configuration and the program controls every move by sliding the blank tile. The goal is to find a sequence of moves from the start state to one fixed goal configuration. On the other hand, Tic-Tac-Toe is a game tree instead of a single-agent state graph: after the AI chooses a move, the opponent also chooses a move, so the next state is not fully controlled by the algorithm. Solving the 8-puzzle means finding a path, preferably the shortest one. Solving Tic-Tac-Toe means choosing a move that is best against an opponent who may also play optimally.

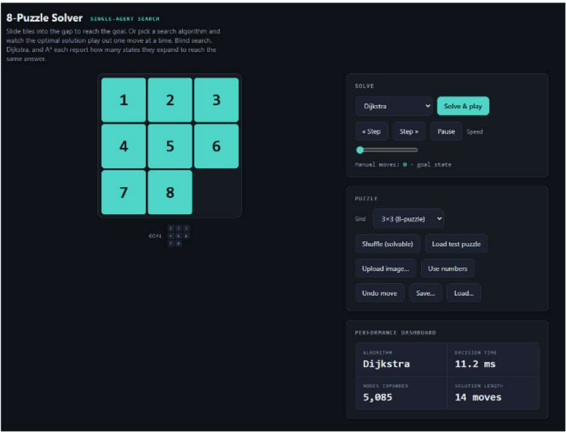
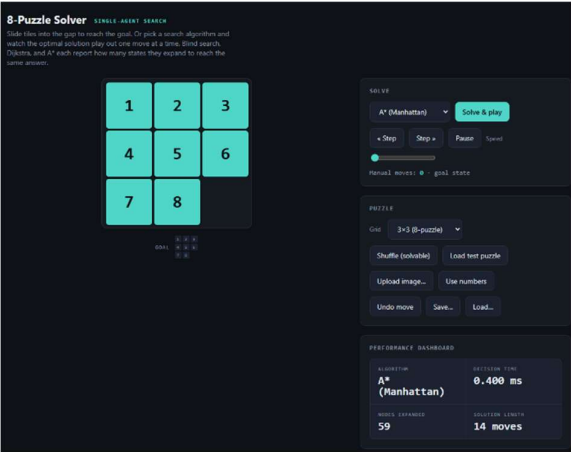
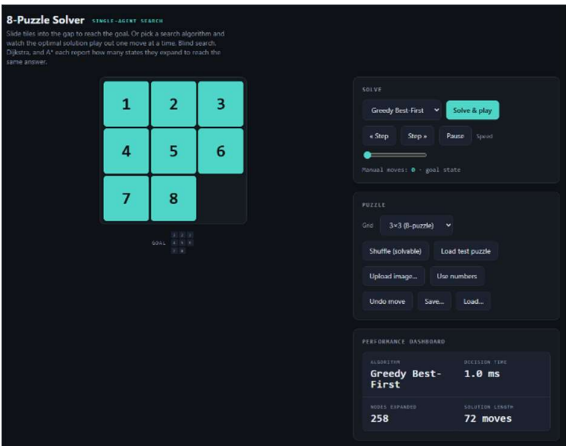
Algorithm Fit:

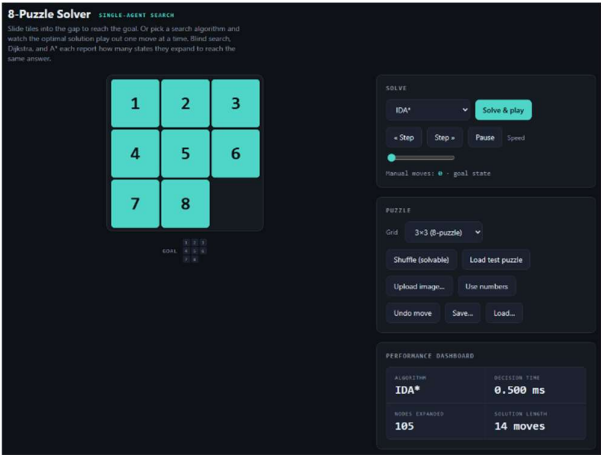
A\* is appropriate for the 8-puzzle because the problem has a clear goal state, a measurable path cost and a useful heuristic such as Manhattan distance. A\* can rank partial solutions by estimated total cost and focus on states that appear closest to the goal. It is not directly applicable to Tic-Tac-Toe because there is no single fixed target board and the value of a move depends on the opponent's future choices. Conversely, Minimax fits Tic-Tac-Toe because it explicitly models alternating maximizing and minimizing players. Minimax does not naturally apply to the 8-puzzle because there is no adversary; every move is chosen by the same solver, so the problem is better modelled as path search.

Empirical Comparison - Module A:

For the standardized puzzle `[[8,1,3],[4,0,2],[7,6,5]]`, the observed results from implementation were as follows:

| Algorithm          | Nodes Expanded | Solution Depth | Decision Time | Screenshot                                                                           |
|--------------------|----------------|----------------|---------------|--------------------------------------------------------------------------------------|
| BFS (Blind Search) | 3,633          | 14             | 6.8 ms        |  |

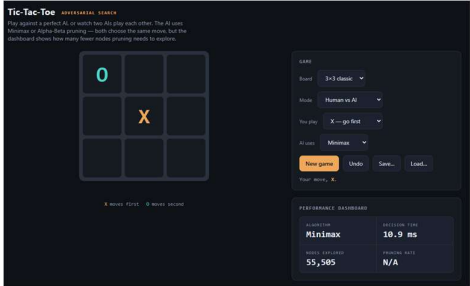
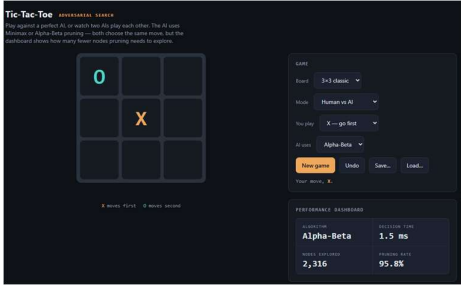
|                            |       |    |         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------------------|-------|----|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dijkstra                   | 5,085 | 14 | 11.2 ms |  The screenshot shows the '8-Puzzle Solver' interface with the 'SOLVE' dropdown set to 'Dijkstra'. The 'PERFORMANCE DASHBOARD' at the bottom right displays the following results: Algorithm: Dijkstra, Decision Time: 11.2 ms, Moves Expanded: 5,085, and Solution Length: 14 moves. The puzzle grid on the left shows tiles 1 through 8 in their initial positions, with the bottom-right cell being empty. |
| A* with Manhattan Distance | 59    | 14 | 0.4 ms  |  The screenshot shows the '8-Puzzle Solver' interface with the 'SOLVE' dropdown set to 'A* (Manhattan)'. The 'PERFORMANCE DASHBOARD' at the bottom right displays the following results: Algorithm: A* (Manhattan), Decision Time: 0.400 ms, Moves Expanded: 59, and Solution Length: 14 moves. The puzzle grid on the left is identical to the first screenshot.                                            |
| Greedy Best First          | 258   | 72 | 1 ms    |  The screenshot shows the '8-Puzzle Solver' interface with the 'SOLVE' dropdown set to 'Greedy Best-First'. The 'PERFORMANCE DASHBOARD' at the bottom right displays the following results: Algorithm: Greedy Best-First, Decision Time: 1.0 ms, Moves Expanded: 258, and Solution Length: 72 moves. The puzzle grid on the left is identical to the first screenshot.                                      |

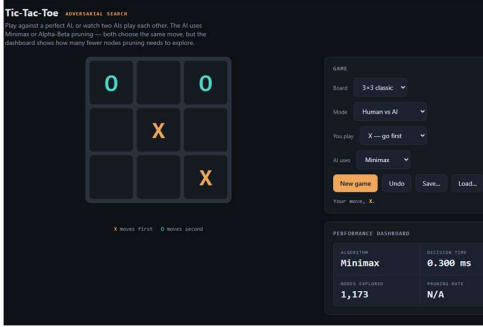
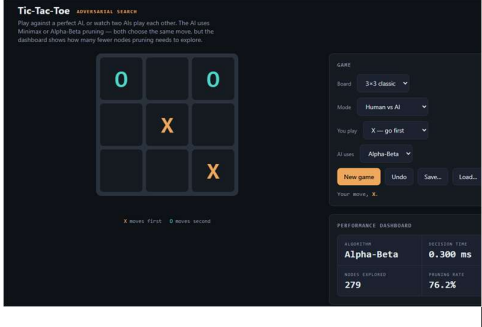
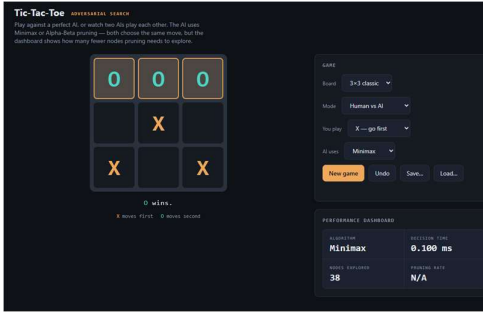
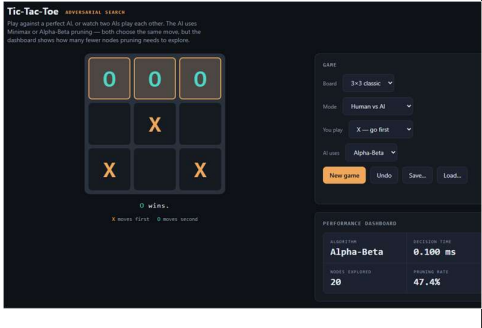
|      |     |    |        |                                                                                    |
|------|-----|----|--------|------------------------------------------------------------------------------------|
| IDA* | 105 | 14 | 0.5 ms |  |
|------|-----|----|--------|------------------------------------------------------------------------------------|

BFS expanded **3,633** nodes and Dijkstra expanded **5,085** nodes, while A\* with Manhattan distance expanded only **59** nodes and still found the same optimal **14-move** solution. In runtime, the same pattern appeared: BFS took **6.8 ms**, Dijkstra took **11.2 ms**, and A\* took only **0.4 ms**. Greedy Best-First expanded **258** nodes in **1.0 ms**, but its solution was **72** moves instead of 14, which shows the cost of using the heuristic without considering path cost. IDA\* expanded **105** nodes in **0.5 ms** and still returned the optimal depth. The surprising result was that A\* was not merely a little better than blind search; on this instance it reduced the search from thousands of nodes to fewer than one hundred.

Empirical Comparison - Module B:

For the standardized Tic-Tac-Toe position, the board was empty and the human played first as X (there is also option to switch). The results below report the same pruning and explored nodes in same human moves.

| Algorithm            | Minimax                                                                             | Alpha-Beta pruning                                                                   |
|----------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| 1 <sup>st</sup> move |  |  |

|                      |                                                                                   |                                                                                    |
|----------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| 2 <sup>nd</sup> move |  |  |
| 3 <sup>rd</sup> move |  |  |

| Algorithm          | Nodes Explored (total) | Pruning in 3 moves  |
|--------------------|------------------------|---------------------|
| Minimax            | 56716                  | 0, 0, 0             |
| Alpha-Beta Pruning | 2615                   | 95.8%, 76.2%, 47.4% |

Across the recorded moves, plain Minimax explored **56,716** total nodes, while Alpha-Beta explored only **2,615** nodes. That is a reduction of **54,101** searched nodes, or about **95.4%** overall using **1 - (2,615 / 56,716)**. Looking at the move-by-move pruning values, Alpha-Beta pruned **95.8%** on the first move, **76.2%** on the second move, and **47.4%** on the third move, while Minimax had **0%** pruning because standard Minimax does not cut off branches. This means Alpha-Beta reached the same kind of decision while avoiding most of the branches that plain Minimax would still examine. The later pruning rates were lower because the remaining game tree becomes smaller and depends strongly on move ordering. This is a useful reminder that Alpha-Beta does not change the theoretical game value; it changes how much unnecessary work is avoided.

Trade-Off Analysis:

BFS is complete for finite branching spaces and optimal when all step costs are equal, but its time and space complexity grow exponentially because it stores and expands entire layers. Dijkstra is complete and optimal with nonnegative costs, but for the 8-puzzle with uniform move costs it behaves similarly to BFS and still uses high time and memory. A\* is complete and optimal when the heuristic is admissible, and it can greatly reduce time in practice, although its memory usage can still be large because it stores frontier states. Greedy Best-First Search is often fast and memory-efficient compared with blind search, but it is not guaranteed to be optimal and can be incomplete in some graph-search settings without repeated-state handling. IDA\* is complete and optimal with

an admissible heuristic while using much less memory than A\*, but it may repeat work across iterations. Minimax is complete and optimal for finite deterministic games with perfect information, but its time complexity is exponential in the game-tree depth. Alpha-Beta has the same completeness and optimality as Minimax, but improves time by pruning irrelevant branches; its space complexity remains similar to depth-first Minimax.

#### **Heuristic Justification**

Manhattan distance is admissible for the 8-puzzle because it counts the minimum number of grid steps each tile must move to reach its goal position while ignoring obstacles and tile interactions. A real move can only move one tile by one square, so the true solution cost cannot be smaller than the sum of these required distances. Therefore, Manhattan distance never overestimates the actual number of moves needed.